KOENIG

step forward



Dependency Injection and IoC

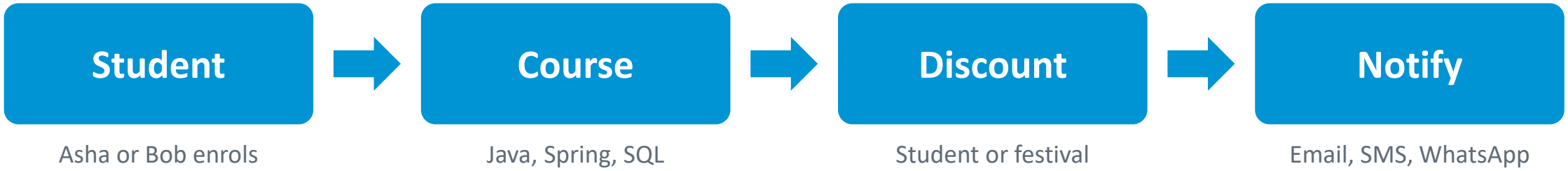
Spring Boot Fundamentals

Dependency Injection and IoC

1. Spring Boot, Gradle, IoC and DI principles
2. The container, bean definitions, scanning and @Bean
3. Constructor, setter and field injection
4. Interfaces, @Qualifier, @Primary and bean collections
5. Bean scopes: singleton, prototype, web
6. Lifecycle callbacks: @PostConstruct, @PreDestroy

Demo Scenario: Learnly Course Enrollment

Learnly Academy sells online courses. Students enrol, get a discount, and receive a notification and a welcome pack. Spring Boot creates and connects every part, so each class only says what it needs.



What the container manages

- **Beans**
- **Injection**
- **Scopes**

What Is Spring Boot?

- **Spring:** a framework whose container creates and connects your objects.
- **Spring Boot:** Spring plus sensible defaults: starters, auto-configuration, embedded server.
- **Run it:** a main method starts everything; no server to install.

```java
@SpringBootApplication
public class LearnlyApp {

    public static void main(String[] args) {
        SpringApplication.run(LearnlyApp.class, args);
    }
}
```

Boot = Spring + defaults: one annotation, one main method

Build Tools: Why We Need One

- **Build tool:** compiles, tests and packages the application.
- **Dependencies:** downloads libraries, and the libraries they need.
- **Maven:** an XML file, pom.xml, with a fixed lifecycle.
- **Gradle:** a script, build.gradle.kts, with tasks and fast incremental builds.

```
// Maven (pom.xml)
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>

// Gradle (build.gradle.kts)
implementation(
    "org.springframework.boot:spring-boot-starter-webmvc")
```

Same job, less typing: this course uses Gradle

Gradle Basics: build.gradle.kts

- **plugins:** java, the Spring Boot plugin and dependency management.
- **repositories:** where libraries come from, usually mavenCentral().
- **dependencies:** implementation lines for the libraries the app needs.
- **Wrapper and tasks:** gradlew uses Gradle 9.8.2; build, test and bootRun are tasks.

```
plugins {
    java
    id("org.springframework.boot") version "4.1.2"
    id("io.spring.dependency-management") version "1.1.7"
}
repositories { mavenCentral() }
dependencies {
    implementation(
        "org.springframework.boot:spring-boot-starter-webmvc")
}
```

Run it: gradlew bootRun Build it: gradlew build

Spring Boot Starters

- **Starter:** one dependency that brings a whole set.
- **Named** spring-boot-starter-<technology>.
- **Boot 4:** modular: webmvc, validation, actuator ...

```
dependencies {
    implementation("org.springframework.boot:spring-boot-starter-webmvc")
    implementation("org.springframework.boot:spring-boot-starter-validation")
    implementation("org.springframework.boot:spring-boot-starter-actuator")
}
```

Boot 4: spring-boot-starter-web is deprecated, use ...-webmvc

Project Structure

- **src/main/java:** your code, in packages.
- **src/main/resources:** application.properties and other files.
- **src/test/java:** your tests.
- **Root package rule:** the main class sits on top; scanning goes down from it.

```
LearnlyEnrollment
|-- build.gradle.kts
|-- settings.gradle.kts
|-- gradlew
`-- src
    `-- main
        |-- java/com/example/learnly
        |   `-- container
        |       |-- ContainerApp.java
        |       `-- CatalogService.java
        `-- resources
            `-- application.properties
```

Beans are found in the package of the main class and below

The Problem: Tight Coupling

- **Hard-wired:** the controller creates its own service.
- **Hard to change:** switching implementation edits the controller.
- **Hard to test:** you cannot replace the service with a fake.
- **Hard to share:** every class builds its own copies.

```
public class QuoteController {

    // this class decides WHICH service
    // and HOW it is built
    private final CatalogService service =
        new SimpleCatalogService();

    public String quote() {
        return service.quote();
    }
}
```

Tight coupling: a class creates what it depends on

Demo: What Tight Coupling Costs

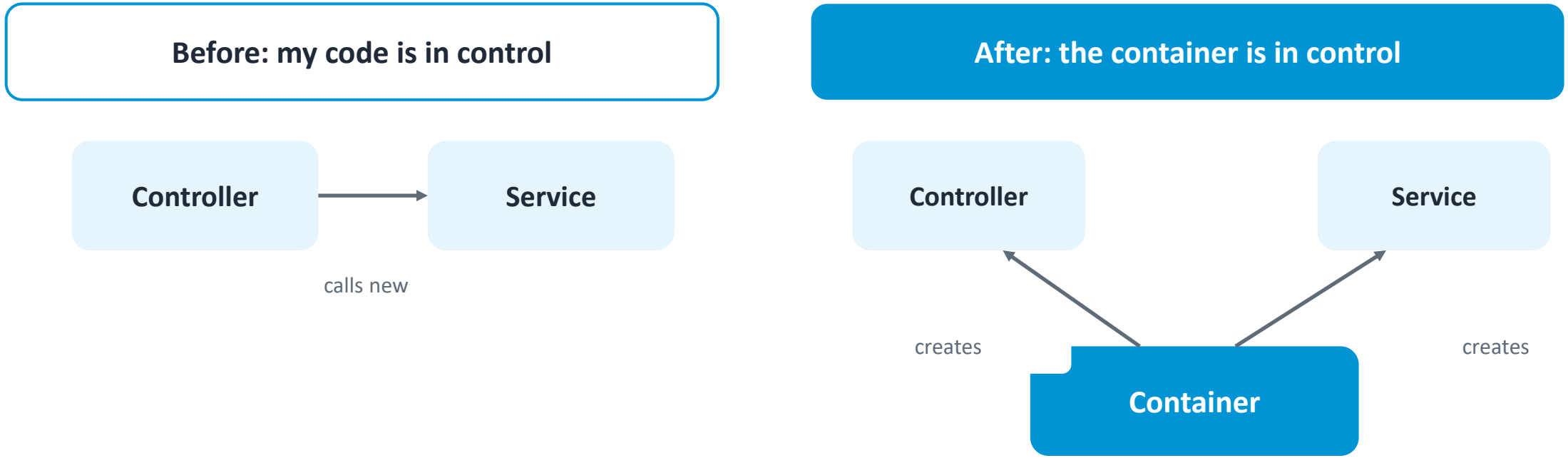
What we build

- **Controller:** an EnrollmentController that builds its own PaymentService.
- **Try to swap:** passing in another service does not compile.
- **Run:** Asha enrols in Java Foundations.

How the concept helps

- **Seeing the pain:** the controller decides which service it uses.
- **Hard to change:** every new payment type means editing the controller.
- **Hard to test:** there is no place to hand in a fake.

Inversion of Control (IoC)



- **IoC:** a principle. Control of creating and connecting objects moves from your code to a framework.
- **Principle:** do not call us, we call you.

Dependency Injection (DI)

- **Dependency:** an object that another object needs.
- **Injection:** the container passes it in from outside.
- **DI** is the most common way to achieve IoC.
- **Your class** only declares WHAT it needs, not how to get it.

```
public class QuoteController {

    private final CatalogService service;

    // the container passes it in
    public QuoteController(CatalogService service) {
        this.service = service;
    }
}
```

Analogy: you order a dish, the kitchen brings the plate

The Spring IoC Container



- **Container:** creates, wires and manages objects.
- **Bean:** an object managed by the container.
- **Plain classes:** no framework base class needed.

ApplicationContext

- **BeanFactory:** the basic container.
- **ApplicationContext:** the full container: adds events, messages, resource loading.
- **Boot creates it** inside SpringApplication.run().
- **run()** returns the context.

```
ConfigurableApplicationContext ctx =
    SpringApplication.run(ContainerApp.class, args);

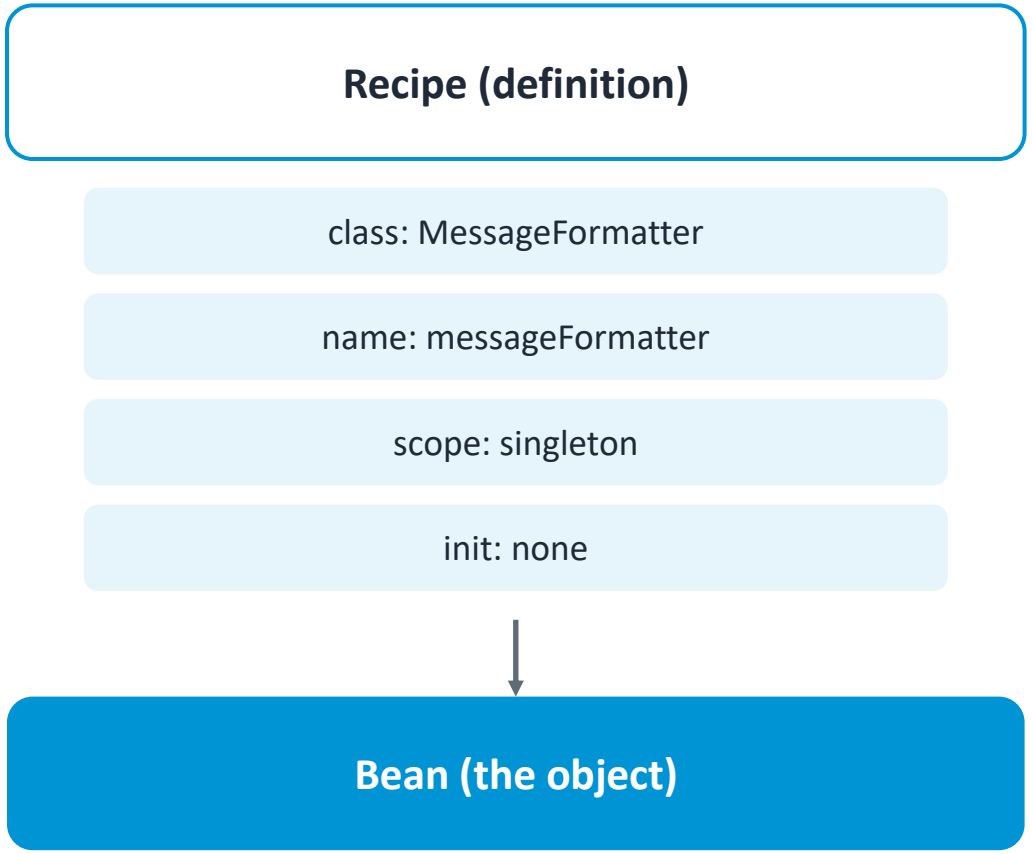
String[] names = ctx.getBeanDefinitionNames();
System.out.println(names.length + " beans");

CatalogService s = ctx.getBean(CatalogService.class);
boolean has = ctx.containsBean("ticketIds");
```

Use getBean in demos and tests; use injection in real code

Bean Definitions

- **A bean definition** is the recipe: class, scope, dependencies, callbacks.
- **The container** reads recipes first, builds objects second.
- **Bean name:** unique id. Default: lower-case class name.
- **Sources:** @Component scan, @Bean methods, auto-configuration.



@Component and Stereotypes

- **@Component:** generic: make a bean of this class.
- **@Service:** business logic.
- **@Repository:** data access (adds exception translation).
- **@Controller / @RestController:** web layer.
- **Same effect,** different meaning for readers and tools.

```
@Component
public class MessageFormatter {
    public String format(String t) {
        return "[" + t.toUpperCase() + "]";
    }
}

@Service
public class SimpleCatalogService
        implements CatalogService { ... }
```

Component Scanning

com.example.learnly.container

- CatalogController
- CatalogService
- CourseRepository
- elsewhere.OutsideService (not scanned)

- **@SpringBootApplication** includes @ComponentScan.
- **Scans** its own package and all sub-packages.
- **Classes outside** that tree are NOT found.
- **Keep** the application class in the root package.

```
@SpringBootApplication   // scan starts here
public class ContainerApp { ... }
```

A bean in another top-level package is silently missing

Demo: Beans from Component Scanning

What we build

- **Three layers:** repository, service and controller, each annotated.
- **ContainerApp:** prints the beans the container created.
- **Boundary:** a class outside the package is not a bean.

How the concept helps

- **Stereotypes:** mark the layer and let the container create the object.
- **Constructor wiring:** a class just declares what it needs.
- **Bean names:** class name with a small first letter.

@Configuration and @Bean

- **@Configuration:** a class that holds bean definitions.
- **@Bean method:** you create the object, the container keeps it.
- **Name:** the method name.
- **Use it for** classes you cannot annotate: libraries, third-party types.
- **Parameters** of a @Bean method are injected too.

```java
@Configuration
public class TicketConfig {

    @Bean
    public TicketIdGenerator ticketIds() {
        return new TicketIdGenerator("TCK-");
    }

    @Bean
    public Report report(TicketIdGenerator ids) {
        return new Report(ids);   // injected
    }
}
```

Demo: @Configuration and @Bean

What we build

- **Run first:** start-up fails because PriceFormatter is not a bean.
- **AppConfig:** a @Bean method for the formatter and one for the Clock.
- **Receipt:** the service prints a dated receipt.

How the concept helps

- **@Bean:** you create the object, the container keeps it.
- **Library classes:** register them without touching their code.
- **Clear errors:** a missing bean stops the start-up with the exact type.

Constructor Injection

- **Dependencies** arrive as constructor parameters.
- **Fields are final:** object is complete from birth.
- **One constructor:** no @Autowired needed.
- **Easy to test:** just call new with a fake.
- **Preferred** for required dependencies.

```java
@RestController
public class QuoteController {

    private final CatalogService quoteService;

    public QuoteController(CatalogService quoteService) {
        this.quoteService = quoteService;
    }

    @GetMapping("/task3/quote")
    public String quote() {
        return quoteService.quote();
    }
}
```

Required dependency? Constructor. Always.

Setter Injection

- **Container calls** a setter after construction.
- **Needs** @Autowired on the setter.
- **Fields can't be final.** Object can be half-built.
- **Fine for** optional dependencies with a default.
- **Otherwise:** avoid.

```
@RestController
public class SetterInjectedController {

    private CatalogService quoteService;

    @Autowired
    public void setCatalogService(CatalogService q) {
        this.quoteService = q;
    }
}
```

Field Injection: Don't Prefer

- **@Autowired on a field:** shortest to type.
- **Hidden:** the constructor does not show the dependencies.
- **Not final,** not immutable.
- **Hard to test:** needs reflection or a Spring context.
- **Hides bad design:** a class with 12 dependencies looks innocent.

```java
// DO NOT COPY
@RestController
public class FieldInjectedController {

    @Autowired
    private CatalogService quoteService;

    // no way to set it without
    // Spring or reflection
}
```

You will see it in old code. Do not write it in new code

Why Constructor Injection Wins

Constructor	final fields, complete object, easy tests, fails at startup if missing
Setter	optional dependencies only, object may be half-built
Field	hidden, mutable, hard to test, the least preferred

Spring team recommendation: constructor injection for required dependencies

Demo: Constructor, Setter and Field Injection

What we build

- **Constructor:** EnrollmentService with a final PaymentGateway.
- **Setter:** an optional CertificateTemplate with required = false.
- **Field:** ReportService, and why new gives a null.

How the concept helps

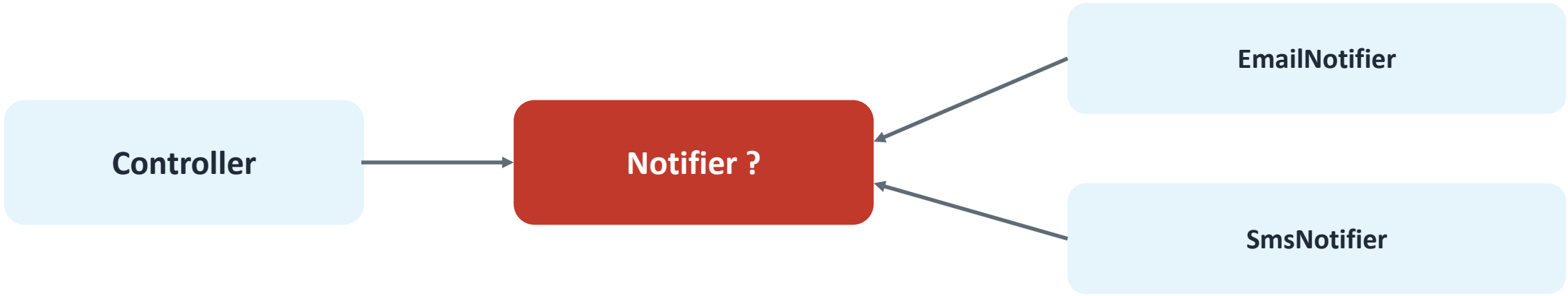
- **Constructor:** complete object, easy to test with a fake.
- **Setter:** optional parts with a safe default.
- **Field:** hidden dependency that breaks without Spring.

Program to Interfaces



- **Depend on** the interface, never the implementation.
- **Swap** implementations without touching callers.
- **Tests** use a fake implementation.
- **One implementation?** Injection by type just works.

Ambiguity: Two Candidates




```
Parameter 0 of constructor in AlertController required
a single bean, but 2 were found:
    - emailNotifier
    - smsNotifier
Action: Consider marking one of the beans as @Primary,
or using @Qualifier to identify the bean ...
```

The container never guesses: it stops at startup

@Qualifier: Choose by Name

- **@Qualifier("name")** selects one specific bean.
- **Name** = bean name by default.
- **Put it** on the injection point.
- **Explicit,** precise, a bit verbose.
- **Can also** label beans with custom qualifiers.

```
@Component
public class SmsNotifier implements Notifier { ... }

@RestController
public class AlertController {

    private final Notifier notifier;

    public AlertController(
            @Qualifier("smsNotifier") Notifier notifier) {
        this.notifier = notifier;
    }
}
```

@Primary: Choose a Default

- **@Primary** marks the default winner.
- **Put it** on the bean, not the consumer.
- **Plain injection** gets the primary.
- **@Qualifier** still beats @Primary.
- **Use it** when one choice is the normal one.

```
@Component
@Primary
public class RegularPrice implements PriceCalculator { ... }

@Component
public class SalePrice implements PriceCalculator { ... }

public PriceController(
        PriceCalculator normal,                    // Regular
        @Qualifier("salePrice") PriceCalculator s) // Sale
{ ... }
```

Demo: Two Candidates, One Winner

What we build

- **DiscountPolicy:** student and festival implementations.
- **Two candidates:** the start-up fails and names both.
- **Fix it:** @Qualifier by name, then @Primary as the default.

How the concept helps

- **Interface:** callers never depend on one implementation.
- **@Primary:** marks the normal choice.
- **@Qualifier:** picks the exception, and wins over @Primary.

Injecting Collections of Beans

- **List<Notifier>:** every bean of that type.
- **@Order:** controls the list order.
- **Map<String,Notifier>:** key = bean name.
- **Great for** the strategy pattern: add a class, no wiring.
- **Empty?** use ObjectProvider or required=false.

```
@RestController
public class BroadcastController {

    private final List<Notifier> all;
    private final Map<String, Notifier> byName;

    public BroadcastController(List<Notifier> all,
            Map<String, Notifier> byName) {
        this.all = all;
        this.byName = byName;
    }
}
// byName keys: emailNotifier, smsNotifier
```

No ambiguity: you asked for all of them

Demo: Injecting Every Notifier

What we build

- **Notifiers:** email, SMS and WhatsApp beans with @Order.
- **List:** NotificationCenter broadcasts to all of them.
- **Map:** pick one channel by bean name.

How the concept helps

- **List of beans:** every bean of the type, in order.
- **Map of beans:** keyed by the bean name.
- **Open for growth:** a new channel needs no wiring.

Bean Scopes

1. **singleton**
   One instance per container. The default.

2. **prototype**
   A new instance on every request to the container.

3. **request**
   One per HTTP request (web only).

4. **session**
   One per HTTP session (web only).

Also: application and websocket scopes exist; we skip them

Singleton vs Prototype

- **Singleton:** created at startup, shared by all, must be thread-safe.
- **Prototype:** created lazily on demand, container forgets it.
- **Singleton** is right for stateless services.
- **Prototype** for stateful, short-lived objects.

```
@Component                          // singleton
public class ConnectionPool { ... }

@Component
@Scope("prototype")                 // new each time
public class Ticket { ... }
```

singleton: ids 1, 1, 1

prototype: ids 2, 3, 4

Request and Session Scope

- **@RequestScope:** new bean for each HTTP request.
- **@SessionScope:** one per session, per browser.
- **Web applications** only.
- **Proxy:** Spring injects a stand-in that finds the real bean.
- **So a singleton** can safely hold them.

```
@Component
@RequestScope
public class RequestStamp { ... }

@Component
@SessionScope
public class VisitCounter {
    private int visits;
    public synchronized int next() {
        return ++visits;
    }
}
```

Trap: Prototype in a Singleton

- **Singleton is built once,** so its prototype is injected once.
- **It never changes.** You expected a new one each call.
- **Fix:** inject ObjectProvider<Ticket>.
- **Or** ApplicationContext.getBean(Ticket.class).

```
// WRONG: always the same Ticket
public StickyHolder(Ticket ticket) { ... }

// RIGHT: ask each time
public ScopeController(ObjectProvider<Ticket> t) {
    this.tickets = t;
}

int id = tickets.getObject().id();   // fresh
Ticket t = ctx.getBean(Ticket.class); // fresh
```

Narrower scope inside wider scope: use ObjectProvider

Demo: Singleton, Prototype and the Trap

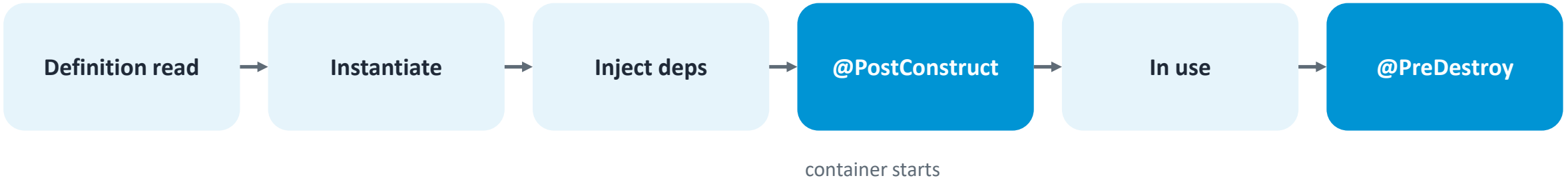
What we build

- **CourseCatalog:** singleton, created once.
- **WelcomePack:** prototype, new on every request.
- **The trap:** a prototype inside a singleton, fixed with ObjectProvider.

How the concept helps

- **Singleton:** shared and stateless, the default.
- **Prototype:** a new object each time it is asked for.
- **ObjectProvider:** asks for a fresh prototype on every call.

The Bean Lifecycle



- **Creation:** constructor, then injection, then @PostConstruct.
- **Destruction:** on context close, @PreDestroy, in reverse dependency order.
- **Prototypes:** created and initialised, but never destroyed by Spring.

@PostConstruct and @PreDestroy

- **@PostConstruct:** runs once dependencies are injected.
- **@PreDestroy:** runs before the bean is thrown away.
- **Package:** jakarta.annotation (Jakarta EE 11).
- **No parameters,** void return.
- **Use for** opening and closing resources.

```
import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;

@Component
public class ConnectionPool {

    @PostConstruct
    void open() { log.info("opening"); }

    @PreDestroy
    void close() { log.info("closing"); }
}
```

Never use the constructor to open resources: use @PostConstruct

Demo: The Bean Lifecycle

What we build

- **EnrollmentLog:** prints construction, injection and @PostConstruct.
- **Work:** two enrolments are recorded.
- **Close:** @PreDestroy flushes the log.

How the concept helps

- **Order:** constructor, injection, then @PostConstruct.
- **@PostConstruct:** a safe place to start work.
- **@PreDestroy:** release resources on shutdown.

Key Takeaways

1 IoC and DI

The container creates and wires objects; classes just declare needs.

2 Register beans

@Component by scanning, @Bean for what you cannot annotate.

3 Constructor first

Final fields; field injection is the one to avoid.

4 Resolve ambiguity

@Qualifier by name, @Primary by default, or inject all.

5 Scopes and lifecycle

Singleton default; prototype via ObjectProvider; @PostConstruct.

6 Boot magic

Starters plus conditional auto-configuration; read the report.



Happy Learning :)